Document number	P2658R0
Date	2022-10-11
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

temporary storage class specifiers

Table of contents

Abstract

“Lifetime issues with references to temporaries can lead to fatal and subtle runtime errors. This applies to both:” [1]

- *“Returned references (for example, when using strings or maps) and”* [1:1]
- *“Returned objects that do not have value semantics (for example using std::string_view).”* [1:2]

This paper proposes the standard adopt storage class specifiers for temporaries in order to provide programmers with tools to manually fix instances of dangling.

Motivating Examples

“Let’s motivate the feature for both, classes not having value semantics and references”, [1:3] by adding 4 new storage class specifiers that are only used by temporaries, such as arguments to functions.

`constinit`	This specifier gives the temporary static storage duration and asserts the following. 1. The parameter is const. 2. The parameter type is a `LiteralType` or a reference to a `LiteralType`. 3. The argument is `constant-initialized`. [2] It is recommended for anything that is `constant-initialized`. [2:1] The `constinit` specifier is a alias for explicit constant initialization i.e. `const static constinit`. The word `constant` may be a better choice.
`variable_scope`	The temporary has the same lifetime of the variable to which it is assigned or `block_scope`, whichever is greater. This specifier is recommended whenever `constinit` can't be used.
`block_scope`	The temporary is scoped to the block that contains said expression. This is the `C` user defined literal lifetime rule. [3] 6.5.2.5 Compound literals This specifier is recommended only for backwards compatibility with the `C` language.
`statement_scope`	The temporary is scoped to the containing full expression. This is the `C++` temporary lifetime rules [2:2]6.7.7 Temporary objects and is the default until one of the other specifiers are applied in which case the other becomes the default until another specifier is given. This specifier is recommended only for backwards compatibility with versions of the `C++` language. It is recommended that programmers transition to using `variable_scope` and `constinit`.

"Classes not Having Value Semantics" [1:4]

"C++ allows the definition of classes that do not have value semantics. One famous example is `std::string_view`: The lifetime of a `string_view` object is bound to an underlying string or character sequence." [1:5]

"Because string has an implicit conversion to `string_view`, it is easy to accidentally program a `string_view` to a character sequence that doesn't exist anymore." [1:6]

"A trivial example is this:" [1:7]

```
std::string_view sv = "hello world"s;// immediate dangling reference
```

It is clear from this `string_view` example that it dangles because `sv` is a reference and `"hello world"s` is a temporary. ***What is being proposed is that same example doesn't dangle just by adding the `constinit` specifier!***

```
std::string_view sv = constinit "hello world"s;
```

If the evaluated constant expression `"hello world"s` had static storage duration just like the string literal `"hello world"` has static storage duration [2:3] *(5.13.5 String literals [lex.string])* then `sv` would be a reference to something that is global and as such would not dangle. This is reasonable based on how programmers reason about constants being immutable variables and temporaries which are known at compile time and do not change for the life of the program.

Dangling *"can occur more indirectly as follows:"* [1:8]

```
std::string operator+ (std::string_view s1, std::string_view s2) {
  return std::string{s1} + std::string{s2};
}
std::string_view sv = "hi";
sv = sv + sv; // fatal runtime error: sv refers to deleted temporary string
```

The problem here is that the lifetime of the temporary is bound to the statement in which it was created, instead of the block that contains said expression.

> `Working Draft, Standard for Programming Language C++` [2:4]
>
> **"*6.7.7 Temporary objects*"**
>
> "***Temporary objects are destroyed as the last step in evaluating the full-expression (6.9.1) that (lexically) contains the point where they were created.*** *This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.*"

Had the temporary been bound to the enclosing block than it would have been alive for at least as long as the returned reference.

```
std::string operator+ (std::string_view s1, std::string_view s2) {
  return std::string{s1} + std::string{s2};
}
std::string_view sv = "hi";
sv = block_scope sv + sv;
```

While this does reduce dangling, it does not eliminate it because if the reference out lives its containing block such as by returning than dangling would still occur. These remaining dangling would at least be more visible as they are usually associated with returns, so you know where to look and if we make the proposed changes than there would be far fewer dangling to look for. It should also be noted that **the current lifetime rules of temporaries are like constants, contrary to programmer's expectations**. This becomes more apparent with slightly more complicated examples.

"Returned References to Temporaries" [1:9]

"Similar problems already exists with references." [1:10]

"A trivial example would be the following:" [1:11]

```
struct X { int a, b; };
int& f(X& x) { return x.a; } // return value lifetime bound to parameter
```

If `f` was called with a temporary than it too would dangle.

```
int& a = f({4, 2});
a = 5; // fatal runtime error
```

If the lifetime of the temporary, `{4, 2}`, was bound to the lifetime of its containing block instead of its containing statement than `a` would not immediately dangle.

```
int& a = f(block_scope {4, 2});
a = 5;
```

Further, `{4, 2}` is constant initialized, so if function `f`'s signature was changed to be `int& f(const X& x)`, since it does not change x, and if `constinit` was added then this example would never dangle.

```
int& a = f(constinit {4, 2});
a = 5;
```

“Class std::string provides such an interface in the current C++ runtime library. For example:” [1:12]

```
char& c = std::string{"hello my pretty long string"}[0];
c = 'x'; // fatal runtime error
std::cout << "c: " << c << '\n'; // fatal runtime error
```

Again, if the lifetime of the temporary, `std::string{"hello my pretty long string"}`, was bound to the lifetime of its containing block instead of its containing statement than `c` would not immediately dangle.

```
char& c = block_scope std::string{"hello my pretty long string"}[0];
c = 'x';
std::cout << "c: " << c << '\n';
```

Further, this more complicated compound temporary expression better illustrates why the current lifetime rules of temporaries are contrary to programmer’s expectations. First of all, let’s rewrite the example, as a programmer would, adding names to everything unnamed.

```
auto anonymous = std::string{"hello my pretty long string"};
char& c = anonymous[0];
c = 'x';
std::cout << "c: " << c << '\n';
```

Even though, the code is the same from a programmer’s perspective, the latter does not dangle while the former do. **Should just naming temporaries, thus turning them into variables, fix memory issues? Should just leaving variables unnamed as temporaries introduce memory issues?** Again, contrary to programmer’s expectations. If we viewed unnecessary/superfluous/immediate dangling as overhead, then the current rules of temporary and constant initialization could be viewed as violations of the zero-overhead principle since just naming temporaries is reasonably written better by hand.

“There are more tricky cases like this. For example, when using the range-base for loop:” [1:13]

```
for (auto x : reversed(make_vector()))
```

“with one of the following definitions, either:” [1:14]

```
template<Range R>
reversed_range reversed(R&& r) {
  return reversed_range{r}; // return value lifetime bound to parameter
}
```

“or” [1:15]

```
template<Range R>
reversed_range reversed(R r) {
  return reversed_range{r}; // return value lifetime bound to parameter
}
```

Yet again, if the lifetime of the temporary, `reversed(make_vector())`, was bound to the lifetime of its containing block instead of its containing statement than `x` would not immediately dangle. Before adding names to everything unnamed, we must expand the range based for loop.

```
{// containing block
  auto&& rg = reversed(make_vector());
  auto pos = rg.begin();
  auto end = rg.end();
  for ( ; pos != end; ++pos ) {
    auto x = *pos;
    ...
  }
}
```

Now, let’s rewrite that expansion, as a programmer would, adding names to everything unnamed.

```
{// containing block
  auto anonymous1 = make_vector();
  auto anonymous2 = reversed(anonymous1);
  auto pos = anonymous2.begin();
  auto end = anonymous2.end();
  for ( ; pos != end; ++pos ) {
    auto x = *pos;
    ...
  }
}
```

Like before, the named version doesn't dangle and as such binding the lifetime of the temporary to the containing block makes more sense to the programmer than binding the lifetime of the temporary to the containing statement. In essence, from a programmer's perspective, **temporaries are anonymously named variables**.

It should be noted too that the current rules of temporaries discourages the use of temporaries because of the dangling it introduces. However, if the lifetime of temporaries was increased to a reasonable degree than programmers would use temporaries more. This would reduce dangling further because there would be fewer named variables that could be propagated outside of their containing scope. This would also improve code clarity by reducing the number of lines of code allowing any remaining dangling to be more clearly seen.

"Finally, such a feature would also help to … fix several bugs we see in practice:" [1:16]

"Consider we have a function returning the value of a map element or a default value if no such element exists without copying it:" [1:17]

```
const V& findOrDefault(const std::map<K,V>& m, const K& key, const V& defvalue);
```

"then this results in a classical bug:" [1:18]

```
std::map<std::string, std::string> myMap;
const std::string& s = findOrDefault(myMap, key, "none"); // runtime bug if key no
```

This example could simply be fixed by adding the `constinit` specifier to the `defvalue` argument.

```
std::map<std::string, std::string> myMap;
const std::string& s = findOrDefault(myMap, key, constinit "none");
```

What if `defvalue` can't be `constant-initialized` because it was created at runtime. If the temporary string's lifetime was bound to the containing block instead of the containing statement than the chance of dangling is greatly reduced and also made more visible. You can say that it **CAN'T** immediately dangle. However, dangling still could occur if the programmer manually propagated the returned value that depends upon the temporary outside of the containing scope.

```
std::map<std::string, std::string> myMap;
const std::string& s = findOrDefault(myMap, key, block_scope not_constexpr());
```

While using the containing's scope instead of the statement's scope is a vast improvement. We can actually do a little bit better. Following is an example of uninitialized and delayed initialization.

```
bool test();

struct X { int a, b; };

constexpr const X* ref2pointer(const X& ref)
{
    return &ref;
}

X x_factory(int a, int b)// not constexpr, runtime construction
{
    return {a, b};
}

int main()
{
  const X* x;// uninitialized
  if(test())
  {
    x = constinit ref2pointer({2, 4});// explicit constant initialization
  }
  else
  {
    // delayed initialization
    //x = statement_scope ref2pointer(x_factory(4, 2));// dangles
    //x = block_scope ref2pointer(x_factory(4, 2));// dangles
    x = variable_scope ref2pointer(x_factory(4, 2));// freed of dangle
  }
}
```

According to this proposal, `constinit ref2pointer({2, 4})` would receive static storage duration. As such that temporary would not dangle.

The variable `x` would dangle if initialized with the expression `statement_scope ref2pointer(x_factory(4, 2))` when the scope is bound to the containing statement. The variable would also dangle if initialized with the expression `block_scope ref2pointer(x_factory(4, 2))` when the scope is bound to the containing block. The variable

would NOT dangle if initialized with the expression `variable_scope ref2pointer(x_factory(4, 2))` when the scope is bound to the lifetime of the variable to which the temporary is assigned, in this case `x`.

It should also be noted that these temporary specifiers are propagated to inner temporaries until they are overridden again. The expression `x_factory(4, 2)` is what needed the specifier but it more convenient for the programmer to put it before the complete temporary expression. Also the specifier applies also to any automatic conversions/initializations performed.

Extending the lifetime of the temporary to be the lifetime of the variable to which it is assigned is not unreasonable for C++. Matter of fact it is already happening but the rules are so restrictive that it limits its use by many programmers as the following examples illustrate.

`Working Draft, Standard for Programming Language C++` [2:5]

"*6.7.7 Temporary objects*"

…

"*5 There are … contexts in which temporaries are destroyed at a different point than the end of the fullexpression.*"

…

"*(6.8)*"

```
template<typename T> using id = T;

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i]; // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
// exactly one of the two temporaries is lifetime-extended
```

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x


struct S {
  const int& m;
};
const S& s = S{1}; // both S and int temporaries have lifetime of s
```

The preceding sections of this proposal is identical at times in wording, in structure as well as in examples to `p0936r0`, the `Bind Returned/Initialized Objects to the Lifetime of Parameters` [1:19] proposal. This shows that similar problems can be solved with simpler solutions, that programmers are already familiar with, such as constants and naming temporaries. It must be conceded that `Bind Returned/Initialized Objects to the Lifetime of Parameters` [1:20] is a more general solution that fixes more dangling while this proposal is more easily understood by programmers of all experience levels but gives programmers tools to fix dangling manually.

Why not just extend the lifetime as prescribed in `Bind Returned/Initialized Objects to the Lifetime of Parameters`?

In that proposal, a question was raised.

“Lifetime Extension or Just a Warning?” “We could use the marker in two ways:”

1. *“Warn only about some possible buggy behavior.”*
2. *“Fix possible buggy behavior by extending the lifetime of temporaries”*

In reality, there are three scenarios; warning, **error** or just fix it by extending the lifetime.

However, things in the real world tend to be more complicated. Depending upon the scenario, at least theoretically, some could be fixed, some could be errors and some could be warnings. Further, waiting on a more complicated solution that can fix everything may never happen or worse be so complicated that the developer, who is ultimately responsible for fixing the code, can no longer understand the lifetimes of the objects created. Shouldn’t we fix what we can, when we can; i.e. low hanging fruit. Also, fixing everything the same way would not even be desirable. Let’s consider a real scenario. Extending one’s lifetime could mean 2 different things.

1. Change automatic storage duration such that a instances’ lifetime is just moved lower on the stack as prescribed in p0936r0.
2. Change automatic storage duration to static storage duration.

If only #1 was applied holistically via p0936r0, `-Wlifetime` or some such, then that would not be appropriate or reasonable for those that really should be fixed by #2. Likewise #2 can’t fix all but DOES make sense for those that it applies to. As such, this proposal and `p0936r0` [1:21] are complimentary.

Personally, `p0936r0` [1:22] or something similar should be adopted regardless because we give the compiler more information than it had before, that a return’s lifetime is dependent upon argument(s) lifetime. When we give more information, like we do with const and constexpr, the

`C++` compiler can do amazing things. Any reduction in undefined behavior, dangling references/pointers and delayed/unitialized errors should be welcomed, at least as long it can be explained simply and rationally.

The work load

The fact is changing every argument of every call of every function is a lot of work and very verbose. In reality, programmers just want to be able to change the default temporary scoping strategy module wide. The following table lists 3 module only attributes which allows the module authors to decide.

`[[default_temporary_scope(variable)]]`	Unless overridden, all temporaries in the module has the same lifetime of the variable to which it is assigned or `block_scope`, whichever is greater. This specifier is the recommended default.
`[[default_temporary_scope(block)]]`	Unless overridden, all temporaries in the module are scoped to the block that contains said expression. This is the `C` user defined literal lifetime rule. [3:1] 6.5.2.5 Compound literals This specifier is recommended only for backwards compatibility with the `C` language.
`[[default_temporary_scope(statement)]]`	Unless overridden, all temporaries in the module are scoped to the containing full expression. This is the `C++` temporary lifetime rules [2:6]6.7.7 Temporary objects and is the default for now for compatibility reasons. This specifier is recommended only for backwards compatibility with the `C++` language. It is recommended that programmers transition to using `[[default_temporary_scope(variable)]]`.

Please note that there was no attribute for `constinit` as this would not be usable. With these module level attributes, all of the specifiers, except `constinit`, could be removed. The `constinit` specifier would still be added to allow the programmer to change an argument in

full or in part to constant static storage duration. Besides being less work and less verbose, module level attribute has the added advantage that this will automatically fix immediate dangling and also greatly reduce any remaining dangling.

In Depth Rationale

There is a general expectation across programming languages that constants or more specifically constant literals are “immutable values which are known at compile time and do not change for the life of the program”. [4] In most programming languages or rather the most widely used programming languages, constants do not dangle. Constants are so simple, so trivial (English wise), that it is shocking to even have to be conscience of dangling. This is shocking to `C++` beginners, expert programmers from other programming languages who come over to `C++` and at times even shocking to experienced `C++` programmers.

Constant Initialization

> `Working Draft, Standard for Programming Language C++` [2:7]
>
> “***6.9.3.2 Static initialization [basic.start.static]***”
>
> “*1 Variables with static storage duration are initialized as a consequence of program initiation. Variables with thread storage duration are initialized as a consequence of thread execution. Within each of these phases of initiation, initialization occurs as follows.*”
>
> “*2 Constant initialization is performed if a variable or temporary object with static or thread storage duration is constant-initialized (7.7).*”

So, how does one perform constant initialization on a temporary with static storage duration and is constant-initialized? It should also be noted that while `static` can be applied explicitly in class data member definition and in function bodies, static isn’t even an option as a modifier to a function argument, so the user doesn’t have a choice and the current default of automatic storage duration instead of static storage duration is less intuitive when constants of constant expressions are involved. In this proposal, I am using the specifier `constinit` as a alias for `const static constinit` . The keyword `constant` would be best. Currently, `constinit` can’t be used on either arguments or local variables, so the existing keyword was just repurposed instead of creating another keyword on our ever growing constant like keyword pile.

Impact on current proposals

p2255r2

A type trait to detect reference binding to temporary [5]

Following is a slightly modified `constexpr` example taken from the `p2255r2` [5:1] proposal. Only the suffix `s` has been added. It is followed by a non `constexpr` example. Currently, such examples are immediately dangling. Via `p2255r2` [5:2], both examples become ill formed. However, with this proposal the examples becomes valid.

	constant
Examples	`std::tuple<const std::string&> x("hello"s);`
Before	`// dangling`
`p2255r2` [5:3]	`// ill-formed`
this proposal only	`// correct` `std::tuple<const std::string&> x(constinit "hello"s);`

	runtime
Examples	`std::tuple<const std::string&> x(factory_of_string_at_runtime());`
Before	`// dangling`
p2255r2 [5:4]	`// ill-formed`
this proposal only	`// well-formed but may dangle latter` `std::tuple<const std::string&> x(variable_scope factory_of_string_at`

With the `constinit` and `variable_scope` specifiers the temporaries cease to be temporaries and instead are just anonymously named variables. They do not have `statement_scope` lifetime that traditional `C++` temporaries have which causes immediate dangling and lead to further dangling.

n3038

Introduce storage-class specifiers for compound literals [6]

In `C23`, the `C` community is getting the comparable feature requested in this proposal, that storage class specifiers can be used on compound literals. This proposal goes beyond by allowing better specifiers to be applied more generally to temporaries.

Present

This proposal should also be considered in the light of the current standards. A better idea of our current rules is necessary to understanding how they may be simplified for the betterment of `C++`.

C Standard Compound Literals

Let's first look at how literals specifically compound literals behave in `C`. There is still a gap between `C99` and `C++` and closing or reducing that gap would not only increase our compatibility but also reduce dangling.

> 2021/10/18 Meneide, C Working Draft [3:2]
>
> *“6.5.2.5 Compound literals”*
>
> **paragraph 5**
>
> *“The value of the compound literal is that of an unnamed object initialized by the initializer list. If the compound literal occurs outside the body of a function, the object has static storage duration; otherwise, it has* ***automatic storage duration associated with the enclosing block****.”*

The lifetime of this “enclosing block” is longer than that of C++. In C++ under 6.7.7 Temporary objects [class.temporary] specifically 6.12 states a *temporary bound to a reference in a new-initializer (7.6.2.8) persists until the completion of the full-expression containing the new-initializer*.

GCC [7] describes the result of this gap.

*“**In C, a compound literal designates an unnamed object with static or automatic storage duration. In C++, a compound literal designates a temporary object that only lives until the end of its full-expression. As a result, well-defined C code that takes the address of a subobject of a compound literal can be undefined in C++**, so G++ rejects the conversion of a temporary array to a pointer.”*

Simply put C has fewer dangling than C++! What is more is that C’s solution covers both const and non const temporaries! Even though it is C, it is more like C++ than what people give this feature credit for because it is tied to blocks/braces, just like RAII. This adds more weight that the C way is more intuitive. Consequently, the remaining dangling should be easier to spot for developers not having to look at superfluous dangling.

GCC even takes this a step forward which is closer to what this proposal is advocating. The last reference also says the following.

*“**As a GNU extension, GCC allows initialization of objects with static storage duration by compound literals (which is not possible in ISO C99 because the initializer is not a constant).** It is handled as if the object were initialized only with the brace-enclosed list if the types of the compound literal and the object match. **The elements of the compound literal must be constant.** If the object being initialized has array type of unknown size, the size is determined by the size of the compound literal.”*

Even the C++ standard recognized that their are other opportunities for constant initialization.

> `Working Draft, Standard for Programming Language C++` [2:8]
>
> **“*6.9.3.2 Static initialization [basic.start.static]*”**
>
> “*3 An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically, provided that*”
>
> “*(3.1) — the dynamic version of the initialization does not change the value of any other object of static or thread storage duration prior to its initialization, and*”
>
> “*(3.2) — the static version of the initialization produces the same value in the initialized variable as would be produced by the dynamic initialization if all variables not required to be initialized statically were initialized dynamically.*”

This proposal is one such opportunity. Besides improving constant initialization, we’ll be increasing memory safety by reducing dangling.

Should `C++` just adopt `C99` literal lifetimes being scoped to the enclosing block instead of to the `C++` statement, in lieu of this proposal?

NO, there is still the expectation among programmers that constants, const evaluations of const-initialized constant expressions, are of static storage duration.

Should `C++` adopt `C99` literal lifetimes being scoped to the enclosing block instead of to the `C++` statement, in addition to this proposal?

YES, `C99` literal lifetimes does not guarantee any reduction in dangling, it just reduces it. This proposal does guarantee but only for const evaluations of constant-initialized constant expressions. Combined their would be an even greater reduction in dangling. As such this proposal and `C99` compound literals are complimentary. The remainder can be mitigated by other measures.

Should `C++` adopt `C99` literal lifetimes being scoped to the enclosing block instead of to the `C++` statement?

YES, the `C++` standard is currently telling programmers that the first two examples in the following table are equivalent with respect to the lifetime of the temporary `{1, 2}` and the named variable `cz`. This is because the lifetime of the temporary `{1, 2}` is bound to the statement, which means it is destroyed before `some_code_after` is called.

Given

```
void any_function(const complex& cz);
```

Programmer code	What `c++` is actually doing!	Programme

`void main()`
`{`
`  some_code_before();`
`  any_function({1, 2});`
`  some_code_after();`
`}`
	`void main()`
	`{`
	`  some_code_before();`
	`  {`
	`  const complex anonymous{1, 2};`
	`  any_function(anonymous);`
	`  }`
	`  some_code_after();`
	`}`
		`void main`
		`{`
		`  some_co`
		`  const c`
		`  any_fun`
		`  some_co`
		`}`

This is contrary to general programmer expectations and how it behaves in `c99`. Besides the fact that a large portion of the `c++` community has their start in `c` and besides the fact that no one, in their right mind, would ever litter their code with superfluous braces for every variable that they would like to be a temporary, their is a more fundamental reason why it is contrary to general programmer expectations. It can actually be impossible to write it that way. Consider another example, now with a return value in which the type does not have a default constructor.

Given

```
no_default_constructor any_function(const complex& cz);
```

Programmer code	`void main()`
	`{`
	`  some_code_before();`
	`  no_default_constructor ndc = any_function({1, 2});`
	`  some_code_after(ndc);`
	`}`

What is `C++` doing?	`void main()`
	`{`
	`  some_code_before();`
	`  {`
	`  const complex anonymous{1, 2};`
	`  no_default_constructor ndc = any_function(anonymous);`
	`  }`
	`  some_code_after(ndc);`
	`}`

What is `C++` doing?	`void main()`
	`{`
	`  some_code_before();`
	`  no_default_constructor ndc;`
	`  {`
	`  const complex anonymous{1, 2};`
	`  ndc = any_function(anonymous);`
	`  }`
	`  some_code_after(ndc);`
	`}`

It should be noted that neither of the “ `What is C++ doing?` ” examples even compile. The first because the variable `ndc` is not accessible to the functional call `some_code_after` . The second because the class `no_default_constructor` doesn’t have a default constructor and as such does not have a uninitialized state. In short, the current `C++` behavior of statement scoping of temporaries instead of containing block scoping is more difficult to reason about because the equivalent code cannot be written by the programmer. As such the `C99` way is simpler, safer and more reasonable. If `C++` is unable to change the lifetimes of temporaries in general then the least it could do is allow programmer’s to set it manually with the `constinit` and `variable_scope` specifiers.

The fundamental flaw

Consider for a moment if the C++ rules were that all variables, named or unnamed/temporaries, ***persists until the completion of the full-expression containing the new-initializer***. [2:9] How useful would that be?

```
auto s = "hello world"s;// immediate dangling reference
std::string_view sv = "hello world"s;// immediate dangling reference
auto reference = some_function("hello world"s);
use_the_ref(reference);// dangle
```

The variable `s` would not be usable. All variables would mostly be immediately dangling. The variable `s` could not be used safely by any statements that follow its initialization. It could not be used safely in nested blocks that follow be that `if`, `for` and `while` statements to name a few. The only place the variable could be used safely if it was anonymously passed as a argument to a function. That would allow multiple statements inside the function call to make use of the instance. If the function returned a reference to the argument or any part of it than there would be further dangling even though it is not unreasonable for a function to return a reference to a portion of or a whole instance, especially when the instance is known to already be alive lower on the stack. In essence, such a rule **divorces the lifetime of the instance from the variable name**. The only use of this from a programmer's perspective is the anonymity of not naming variables as a form of access control. In short, programmers could not program. Doesn't this sound familiar, for it is our current temporary lifetime rule!

Now, consider for a moment if the C++ rules were that all variables that do not have static storage duration, has automatic storage duration associated with the enclosing block of the expression as if the compiler was naming the temporaries anonymously or associated with the enclosing block of the variable to which the initialization is assigned, whichever is greater lifetime. How useful would that be?

```
auto s = "hello world"s;
std::string_view sv = "hello world"s;
auto reference = some_function("hello world"s);
use_the_ref(reference);
```

The variable `s` would be usable. No variables would immediately dangle. The variable `s` could be used safely by any statements that follow its initialization. It could be used safely in nested blocks that follow be that `if`, `for` and `while` statements to name a few. By default, the variable could be used safely when anonymously passed as a argument to a function. If the function returned a reference to the argument or any part of it than there would not be further dangling unless the developer manually propagated the reference lower on the stack such as with a return. Even the benefit of anonymity when using temporaries are not lost and the longer

lifetime doesn't impact other instances that don't even have access to said temporary. In short, programmers are freed from much dangling. Further, much the remaining dangling coalesces around returns and yields.

Until the day when `C++` can change the lifetime of temporaries, it would be nice if programmer's had the ability to change the lifetime.

```
auto s = constinit "hello world"s;
std::string_view sv = constinit "hello world"s;
auto reference = some_function(variable_scope "hello world"s);// yes, constinit is
use_the_ref(reference);
```

Outstanding Issue

CWG900 Lifetime of temporaries in range-based for

```
// some function

std::vector<int> foo();

// correct usage

auto v = foo();

for( auto i : reverse(v) ) { std::cout << i << std::endl; }

// problematic usage

for( auto i : reverse(foo()) ) { std::cout << i << std::endl; }
```

With `C99` literal enclosing block lifetime, this example would not dangle. Let's fix this with `variable_scope`.

```
// some function

std::vector<int> foo();

// correct usage

for( auto i : variable_scope reverse(foo()) ) { std::cout << i << std::endl; }
```

In the identifying paper for this issue, `Fix the range-based for loop, Rev1` [8], says the following:

"***The Root Cause for the problem***"

"*The reason for the undefined behavior above is that according to the current specification, the range-base for loop internally is* ***expanded to multiple statements****:*"

- "*First, we have some initializations using the for-range-initializer after the colon and*"
- "*Then, we are calling a low-level for loop*"

While certainly a factor, the problem is **NOT** that internally, the range-base for loop is expanded to multiple statements. It is rather that one of those statements has a scope of the statement instead of the scope of the containing block. The scoping difference between `C99` and `C++` rears it head again. From the programmers perspective, the issue in both cases is that `C++` doesn't treat temporaries, unnamed variable as if they were named by the programmer just anonymously. The supposed `correct usage` highlights this fact.

```
auto v = foo();

for( auto i : reverse(v) ) { std::cout << i << std::endl; }
```

If you just name it, it works! Had `reverse(foo())` been scoped to the block that contains the range based for loop than this too would have worked.

Should have worked	**`C99` would have worked**	**Progr**

`{// containing block`
`  for( auto i : reverse(foo()) )`
`  {`
`  std::cout << i << std::endl;`
`  }`
`}`
	`{// containing block`
	`  auto&& rg = reverse(foo());`
	`  auto pos = rg.begin();`
	`  auto end = rg.end();`
	`  for ( ; pos != end; ++pos ) {`
	`  int i = *pos;`
	`  ...`
	`  }`
	`}`
		`{//`
		`  a`
		`  a`
		`  f`
		`  {`
		`  }`
		`}`

It should be no different had the programmer broken a compound statement into it's components and named them individually.

Other Anonymous Things

The pain of immediate dangling associated with temporaries are especially felt when working with other anonymous language features of `C++` such as lambda functions and coroutines.

Lambda functions

Whenever a lambda function captures a reference to a temporary it immediately dangles before an opportunity is given to call it, unless it is a immediately invoked lambda/function expression.

```
[&c1 = "hello"s](const std::string& s)// OK
{
    return c1 + " "s + s;
}("world"s);// immediately invoked lambda/function expression

auto lambda = [&c1 = "hello"s](const std::string& s)// immediate dangling
{
    return c1 + " "s + s;
}
// ...
lambda("world"s);
```

This problem is resolved when the scope of temporaries is to the enclosing block instead of the containing expression.

```
auto lambda = [&c1 = variable_scope "hello"s](const std::string& s)
{
    return c1 + " "s + s;
}
// ...
lambda("world"s);
```

This is the same had the temporary been named.

```
auto anonymous = "hello"s;
auto lambda = [&c1 = anonymous](const std::string& s)
{
    return c1 + " "s + s;
}
// ...
lambda("world"s);
```

This specific immediately dangling example is also fixed by explicit constant initialization.

```
auto lambda = [&c1 = constinit "hello"s](const std::string& s)
{
    return c1 + " "s + s;
}
// ...
lambda("world"s);
```

Coroutines

Given

```
generator<char> each_char(const std::string& s) {
    for (char ch : s) {
        co_yield ch;
    }
}
```

Similarly, whenever a coroutine gets constructed with a reference to a temporary it immediately dangles before an opportunity is given for it to be `co_await` ed upon.

```
int main() {
    for (char ch : each_char("hello world")) {// immediate dangling
        std::print(ch);
    }
}
```

This problem is also resolved when the scope of temporaries is to the enclosing block instead of the containing expression.

```
int main() {
    for (char ch : each_char(variable_scope "hello world")) {
        std::print(ch);
    }
}
```

This also is the same had the temporary been named.

```
int main() {
    auto s = "hello world"s;
    for (char ch : each_char(s)) {
        std::print(ch);
    }
}
```

This specific immediately dangling example also is also fixed by explicit constant initialization.

```
int main() {
    for (char ch : each_char(constinit "hello world")) {
        std::print(ch);
    }
}
```

Tooling Opportunities

There area a couple tooling opportunities especially with respect to the `constinit` specifier.

- A command line and/or IDE tool could analyze the code for `const`, `constexpr` / `LiteralType` and constant-initialized and if the conditions matches automatically add the `constinit` specifier for code reviewers.
- Another command line and/or IDE tool could strip `constinit` specifier from any temporaries for programmers.

Combined they would form a `constinit` toggle which wouldn't be all that much different from whitespace and special character toggles already found in many IDE(s).

Summary

There are a couple of principles repeated throughout this proposal.

1. Constants really should have static storage duration so that they never dangle.
2. Temporaries are expected to be just anonymously named variables / `C99` compound literals lifetime rule
    - **variable scope**: is better than block scope for fixing dangling throughout the body of a function

The advantages to `C++` with adopting this proposal is manifold.

`constinit`, `variable_scope`, `block_scope` and `statement_scope` specifiers	**`constinit` specifier and `[[default_temporary_scope(variable)]]`**
• Reduce the gap between `C++` and `C99` compound literals • Reduce the gap between `C++` and `C23` storage-class specifiers • Improve the potential contribution of `C++` 's new specifiers back to `C` • Increase and improve upon the utilization of ROM and the benefits that entails • **Empower programmers to be able to fix most dangling simply**	• Reduce the gap between `C++` and `C99` compound literals • Reduce the gap between `C++` and `C23` storage-class specifiers • Improve the potential contribution of `C++` 's new specifier back to `C` • Increase and improve upon the utilization of ROM and the benefits that entails • **Empower programmers to be able to fix most dangling simply with a whole lot less verbosity** • **Automatically eliminate immediate dangling** • **Automatically reduce all remaining dangling** • **Automatically reduce unitialized and delayed initialization errors**

Frequently Asked Questions

What about locality of reference?

It is true that globals can be slower than locals because they are farther in memory from the code that uses them. So let me clarify, when I say `static storage duration`, I really mean **logically** `static storage duration`. If a type is a `PODType` / `TrivialType` or `LiteralType` than there is nothing preventing the compiler from copying the global to a local that is closer to the executing code. Rather, the compiler **must** ensure that the instance is always available; **effectively** `static storage duration`.

Consider this from an processor and assembly/machine language standpoint. A processor usually has instructions that works with memory. Whether that memory is ROM or is logically so because it is never written to by a program, then we have constants.

```
mov <register>,<memory>
```

A processor may also have specialized versions of common instructions where a constant value is taken as part of the instruction itself. This too is a constant. However, this constant is guaranteed closer to the code because it is physically a part of it.

```
mov <register>,<constant>
mov <memory>,<constant>
```

What is more interesting is these two examples of constants have different value categories since the ROM version is addressable and the instruction only version, clearly, is not. It should also be noted that the later unnamed/unaddressable version physically can't dangle.

Is `variable_scope` easy to teach?

values	pointers with `C99` &	references with `C++`

`int i = 5;`
`if(whatever)`
`{`
`  i = 7;`
`}`
`else`
`{`
`  i = 9;`
`}`
`// use i`
	`int* i = &5;// or uninitialized`
	`if(whatever)`
	`{`
	`  i = variable_scope &7;`
	`}`
	`else`
	`{`
	`  i = variable_scope &9;`
	`}`
	`// use i`
		`std::reference_wrapper<`
		`if(whatever)`
		`{`
		`  i = std::ref(variable`
		`}`
		`else`
		`{`
		`  i = std::ref(variable`
		`}`
		`// use i`

In the `values` example, there is no dangling. Programmers trust the compiler to allocate and deallocate instances on the stack. They have to because the programmer has little to no control over deallocation. With the current `C++` statement scope rules or the `C99` block scope rule, both the `pointers` and `references` examples dangle. In other words, the compilers who are primarily responsible for the stack has rules that needlessly causes dangling and embarrassing worse, immediate dangling. This violates the programmer's trust in their compiler. Variable scope is better because it restores the programmer's trust in their compiler/language by causing temporaries to match the value semantics of variables. Further, it avoids dangling throughout the body of the function whether it is anything that introduces new blocks/scopes be that `if`, `switch`, `while`, `for` statements and the nesting of these constructs.

How do these specifiers propagate?

These specifiers apply to the temporary immediately to the right of said specifier and to any child temporaries. It does not impact any parent or sibling temporaries. Consider these examples:

```
// all of the temporaries has the default temporary scope as
// specified by the module attribute otherwise statement scope
f({1, { {2, 3}, 4}, {5, 6} });
// only 4 has constinit scope
f({1, { {2, 3}, constinit 4}, {5, 6} });
// only {2, 3}, 2, 3 has constinit scope
f({1, { constinit {2, 3}, 4}, {5, 6} });
// only { {2, 3}, 4}, {2, 3}, 2, 3, 4 has constinit scope
f({1, constinit { {2, 3}, 4}, {5, 6} });
// all of the arguments have has constinit scope
f(constinit {1, { {2, 3}, 4}, {5, 6} });
// only 5 has constinit scope
f({1, { {2, 3}, 4}, {constinit 5, 6} });
```

Doesn’t this make C++ harder to teach?

Until the day that all dangling gets fixed, any new tools to assist developer’s in fixing dangling would still require programmers to be able to identify any dangling and know how to fix it specific to the given scenario, as there are multiple solutions. Since dangling occurs even for things as simple as constants and immediate dangling is so naturally easy to produce than dangling resolution still have to be taught, even to beginners.

So, what do we teach now and what bearing does these teachings, the `C++` standard and this proposal have on one another.

C++ Core Guidelines
F.42: Return a `T*` to indicate a position (only) [9]
Note *Do not return a pointer to something that is not in the caller’s scope; see F.43.* [10]

Returning references to something in the caller’s scope is only natural. It is a part of our reference delegating programming model. A function when given a reference does not know how the instance was created and it doesn’t care as long as it is good for the life of the function call and beyond. Unfortunately, scoping temporary arguments to the statement instead of the containing block doesn’t just create immediate dangling but it provides to functions references to instances that are near death. These instances are almost dead on arrival. Having the ability to return a reference to a caller’s instance or a sub-instance thereof assumes, correctly, that reference from the caller’s scope would still be alive after this function call. The fact that

temporary rules shortened the life to the statement is at odds with what we teach. This proposal allows programmers to restore to temporaries the lifetime of anonymously named variables which is not only natural but also consistent with what programmers already know. It is also in line with what we teach, as was codified in the C++ Core Guidelines.

References

1. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf) ↩

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf) ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩

3. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf) ↩ ↩ ↩

4. https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constants (https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constants) ↩

5. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2255r2.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2255r2.html) ↩ ↩ ↩ ↩ ↩

6. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3038.htm (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3038.htm) ↩

7. https://gcc.gnu.org/onlinedocs/gcc/Compound-Literals.html (https://gcc.gnu.org/onlinedocs/gcc/Compound-Literals.html) ↩

8. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r1.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r1.pdf) ↩

9. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only) ↩

10. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object) ↩